

01_generate_initial_data

August 31, 2026

1 01 – Generate 2D initial data

This notebook reproduces the ini_dat_v2.ipynb workflow using the refactored PIC module.

```
[ ]: using Pkg
      Pkg.activate(joinpath(@__DIR__, ".."))
      Pkg.instantiate()
```

```
[ ]: cd(@__DIR__)
      include(joinpath("../src", "PIC.jl"))
      using .PIC
      using Plots
      using Statistics
      using FFTW
      using LinearAlgebra

      const N_exp = 5
      const N = 10^N_exp
      const order = 5
      const factor = 2

      J = (50, 50)
      box = Box((0.0, 1.0, 0.0, 1.0))
      nm = [1, 1]
      alpha_exp = 1
      = 10.0^(-alpha_exp)
      _exp = 3
      = 10.0^(-_exp)
      p_max = sqrt((1 + 10)^2 - 1)
      box_p = Box((-p_max, p_max, -p_max, p_max))

      data_name = ␣
      ↪ "par_dis_rel_thermal_nm_[$(nm[1]),$(nm[2])]_alp$(alpha_exp)_N$(N_exp)_Th$( _exp) "
      par_f_x = ( , nm, box)
      par_f_p = ( , 2)

      par_dis = build_initial_data_D(
        joinpath("../", "Initial_Distributions", data_name),
```

```

    N,
    f_x, f_x_max, par_f_x, box,
    f_p_thermal, f_p_thermal_max, par_f_p, box_p
)

println("Saved to: Initial_Distributions/$(data_name).jld2")
println("length(par_dis) = ", length(par_dis))

```

1.1 Reshape into (N, 4) matrix for convenience

```

[ ]: # Reshape into N×4: [x, y, px, py]

r = reshape(par_dis, (4, N))'
x = r[:, 1]
y = r[:, 2]
px = r[:, 3]
py = r[:, 4]

```

1.2 Diagnostic checks

```

[ ]: ## Diagnostic checks

# Build a minimal state so we can use module diagnostics
g_dummy = PICGrid(box, J, order)
u_dummy = make_state(g_dummy, N)
u_dummy[1:4N] .= par_dis

# Temperatures
T_rel = temperature(u_dummy, g_dummy, N; mode=:rel)
T_norel = temperature(u_dummy, g_dummy, N; mode=:norel)
println("relativistic temperature T = ", T_rel, " (target = ", , ")")
println("non-relativistic temperature T = ", T_norel, " (target /2 = ", /2, "
↪)")

# Total momentum (should be ~0 because symmetric pairs were generated)
P_total = total_momentum(u_dummy, g_dummy, N)
println("total momentum P = ", P_total)
println("|P|/N = ", norm(P_total) / N)

println("x range: ", extrema(x))
println("y range: ", extrema(y))
println("<px^2> = ", mean(px.^2), " ( /2 = ", /2, ")")
println("<py^2> = ", mean(py.^2), " ( /2 = ", /2, ")")

```

1.3 Spatial plots

```
[ ]: ## Spatial plots

# 2D histogram of particle positions
h2 = histogram2d(x, y, bins=50,
    title="spatial distribution", c=:thermal, aspectratio=1,
    xlabel="x", ylabel="y")

# 1D projection along x and comparison with analytic profile (to check this use
    ↪ nm = [n, 0])
hx = histogram(x, bins=50, normalize=true, label="sampled", title="density
    ↪ profile along x")
kx = 2 * nm[1] / extent(box, 1)
analytic_x = xx -> (1 + * cos(kx * xx)) / extent(box, 1)
xx = range(box.lo[1], box.hi[1], length=200)
plot!(hx, xx, analytic_x.(xx), lw=2, label="analytic")

# 1D projection along y and comparison with analytic profile (to check this use
    ↪ nm = [0, m])
hy = histogram(y, bins=50, normalize=true, label="sampled", title="density
    ↪ profile along y")
ky = 2 * nm[2] / extent(box, 2)
analytic_y = yy -> (1 + * cos(ky * yy)) / extent(box, 2)
yy = range(box.lo[2], box.hi[2], length=200)
plot!(hy, yy, analytic_y.(yy), lw=2, label="analytic")

plot(h2, hx, hy, layout=(1, 3), size=(1200, 350))
```

1.4 Momentum plots

```
[ ]: ## Momentum plots

# 2D histogram of momenta
mp2 = histogram2d(px, py, bins=50,
    title="momentum distribution", c=:thermal, aspectratio=1,
    xlabel="px", ylabel="py")

# 1D histograms with analytic thermal profile
analytic_p = pp -> exp(-pp.^2 / / 2) / sqrt( * * 2)
pp = range(-p_max, p_max, length=200)

hpx = histogram(px, bins=50, normalize=true, label="sampled", title="px
    ↪ distribution")
plot!(hpx, pp, analytic_p.(pp), lw=2, label="analytic")
```

```

hpy = histogram(py, bins=50, normalize=True, label="sampled", title="py
↪distribution")
plot!(hpy, pp, analytic_p.(pp), lw=2, label="analytic")

plot(mp2, hpx, hpy, layout=(1, 3), size=(1200, 350))

```

1.5 Scatter plots of the first few thousand particles

```

[ ]: ## Scatter plots of the first few thousand particles

nplot = min(N, 4000)
scatter(x[1:nplot], y[1:nplot],
        title="positions (first $nplot particles)", aspectratio=1,
        xlabel="x", ylabel="y", markersize=1, label=false)

scatter(px[1:nplot], py[1:nplot],
        title="momenta (first $nplot particles)", aspectratio=1,
        xlabel="px", ylabel="py", markersize=1, label=false)

```